

Learning-Based Nonlinear MPC With Integrated Moving Horizon Estimation of Quadrotors

Chenjing Meng , Graduate Student Member, IEEE, and Huiping Li , Senior Member, IEEE

Abstract—This article studies a learning-based cascade nonlinear model predictive control (NMPC) problem for quadrotor trajectory tracking under unknown time-varying disturbances. A nonlinear moving horizon estimation (NMHE) method is employed to estimate the quadrotor's states and key parameters such as aerodynamic coefficients, mass, and moment of inertia. Based on the estimates, a differentiable NMPC, along with an online learning method to adjust the cost function considering the tracking trajectory, is developed. This enables the framework to adapt to dynamic environments. Finally, comparative studies validate the effectiveness and advantages of the proposed method.

Index Terms—Learning-based control, nonlinear model predictive control (NMPC), nonlinear moving horizon estimate, quadrotor, trajectory tracking.

I. INTRODUCTION

QUADROTORS have attracted considerable attention in recent years due to their inherent capability, agility, and cost-effectiveness. These characteristics have facilitated widespread implementation across various industries, including search and rescue, precise agriculture, public safety, and disaster response. Despite these advantages, controlling quadrotors to execute complex tasks with high precision poses challenges due to their nonlinear dynamics, variations in model parameters, and limited actuation capabilities. Moreover, time-varying dynamics and environmental disturbances, such as aerodynamic effects [1] and unpredictable wind conditions [2], further complicate the control problem.

Model predictive control (MPC) has proven effective for constrained systems, including quadrotors [3]. While linear MPC is commonly used, its performance often degrades when addressing the nonlinear dynamics of quadrotors during aggressive maneuvers. Nonlinear MPC (NMPC), which offers

superior control for complex tasks such as load transportation [4], collision avoidance [5], and precise vertical takeoff and landing operations [6], has gained popularity with the advancement of computational power. However, the success of model-based controllers heavily depends on the accuracy of the system model, and deviations in parameters can lead to suboptimal performance. This issue is particularly problematic in quadrotors, where manually tuning controllers for predefined tasks often fails in new scenarios. To address this problem, the controller must adapt to changing conditions by learning and adjusting in real time.

Recent advances have integrated learning-based methods into MPC frameworks to enhance quadrotor control. Techniques such as regression models [7], reinforcement learning (RL) [8], deep learning [9], and Gaussian processes (GPs) [10] have been explored for system modeling. Among these, the deep neural networks (DNNs) effectively approximate dynamic models, replacing nominal system dynamics in MPC [11], [12]. However, DNNs often face challenges like local minima, which can degrade control performance. Similarly, GP provides uncertainty insights but struggles with real-time adaptation due to its reliance on large datasets.

To address these limitations, differentiable MPC (DMPC) has emerged as a promising method, effectively bridging optimal control and RL [13]. By leveraging the differentiability of control policies, DMPC facilitates efficient optimization and adaptive adjustments. However, its open-loop design limits its ability to adapt to dynamic and uncertain environments [14]. Furthermore, robust learning in such contexts demands precise estimation of system parameters, such as mass and aerodynamic characteristics, particularly in the presence of persistent, time-varying disturbances.

In addition, the accurate state estimation is also essential for quadrotor control, especially when dealing with unknown model parameters and disturbances. Extended Kalman filter (EKF) [15] and unscented Kalman filter (UKF) [16] are commonly used but may struggle with non-Gaussian disturbances. Conversely, moving horizon estimation (MHE) [17], [18] has demonstrated superiority in addressing nonlinear estimation problems with non-Gaussian disturbances [19] and handling constraints by leveraging a series of historical measurements over a fixed-length horizon, offering better robustness and accuracy [20], [21].

Motivated by this fact, this article presents a novel learning-based nonlinear MHE (NMHE)-DNMPC framework

Received 28 September 2024; revised 24 December 2024; accepted 26 January 2025. Date of publication 27 February 2025; date of current version 23 July 2025. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant U22B2039, Grant 62273281, and Grant U24B20183. (Corresponding author: Huiping Li.)

The authors are with the School of Marine Science and Technology, Northwestern Polytechnical University, Xi'an 710072, China (e-mail: cjmeng@mail.nwpu.edu.cn; lihuiping@nwpu.edu.cn).

Digital Object Identifier 10.1109/TIE.2025.3541282

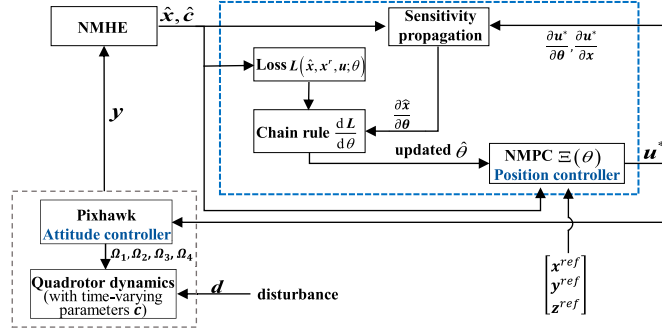


Fig. 1. Overview of the learning-based cascade approach.

for quadrotor control under time-varying disturbances and unknown parameters (see Fig. 1). The proposed framework integrates NMHE for estimating the full state vector, external disturbances, and key model parameters such as mass, thrust coefficient, and moment of inertia. These real-time estimates provide essential information for the differentiable NMPC (DNMPC), which adjusts its cost function using a trajectory-dependent learning mechanism. To achieve this, we employ the DiffTune method [22] for closed-loop auto-tuning of controller parameters, enhancing tracking performance and ensuring adaptive trajectory tracking. The effectiveness of the proposed method is verified with a Crazyflie quadrotor.

The main contributions of this article are as follows.

- 1) We propose a learning-based NMHE-DNMPC framework that combines real-time estimation (via NMHE) of states, disturbances, and model parameters with adaptive control for quadrotor trajectory tracking, making it effective in partially observable and dynamical scenarios.
- 2) The framework employs a trajectory-dependent learning mechanism within the DNMPC, using the DiffTune method for closed-loop auto-tuning, significantly improving tracking performance under time-varying disturbances.
- 3) The effectiveness of the proposed framework is validated through detailed simulation experiments with a Crazyflie quadrotor, demonstrating its robustness against time-varying disturbances and parameter uncertainties.

The article is organized as follows. In Section II, the dynamic modeling of the quadrotor system is introduced. Section III presents the formulation of NMHE and DNMPC. In Section IV, the learning-based NMHE-DNMPC framework is described in detail, including the algorithm. Comparative tests and studies are discussed in Section V. Finally, a summary of the findings is presented in Section VI.

II. PRELIMINARIES: QUADROTOR DYNAMICS

We model the quadrotor UAV as a rigid body with six degrees of freedom shown in Fig. 2, and its dynamics can be represented as

$$\begin{aligned}\dot{\mathbf{p}}_E &= \mathbf{v}_E, \\ \dot{\mathbf{v}}_E &= m^{-1} (-mge_3 + \mathbf{R}_{BE}F\mathbf{e}_3 + \mathbf{d}_f)\end{aligned}$$

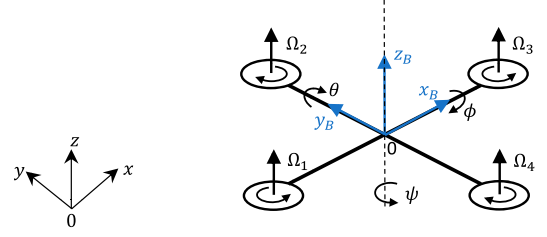


Fig. 2. Schematic diagram of a quadrotor UAV.

$$\begin{aligned}\dot{\mathbf{q}}_{BE} &= \frac{1}{2} \begin{bmatrix} \mathbf{0} & -\boldsymbol{\omega}_B^\top \\ \boldsymbol{\omega}_B & -\boldsymbol{\omega}_B^\times \end{bmatrix} \mathbf{q}_{BE} \\ \dot{\boldsymbol{\omega}}_B &= \mathbf{J}_B^{-1} (-\boldsymbol{\omega}_B^\times \mathbf{J}_B \boldsymbol{\omega}_B + \boldsymbol{\tau} + \mathbf{d}_\tau)\end{aligned}\quad (1)$$

where m denotes the quadrotor's mass, and $\mathbf{J}_B \in \mathbb{R}^{3 \times 3}$ represents the moment of inertia, corresponding to I_{xx} , I_{yy} , and I_{zz} about x-, y-, and z-axis, respectively. The angular velocity in the body frame $\mathcal{F}_B$ is denoted by $\boldsymbol{\omega}_B \in \mathbb{R}^3$, while $\mathbf{p}_E \in \mathbb{R}^3$ and $\mathbf{v}_E \in \mathbb{R}^3$ represent the global central-of-mass (CoM) position and velocity relative to the earth frame $\mathcal{F}_E$, respectively. Additionally, $\mathbf{q}_{BE}$ denotes the quaternion representing the rotation from the body frame $\mathcal{F}_B$ to the earth frame $\mathcal{F}_E$, and $\mathbf{q}_{BE}$ represents the rotational matrix from $\mathcal{F}_B$ to $\mathcal{F}_E$. The disturbance forces $\mathbf{d}_f \in \mathbb{R}^3$ and torques $\mathbf{d}_\tau \in \mathbb{R}^3$ are defined in $\mathcal{F}_E$ and $\mathcal{F}_B$, respectively. The acceleration due to gravity is g and the unit vector in the vertical direction is $\mathbf{e}_3 = [0, 0, 1]^\top$. The skew-symmetric matrix $\boldsymbol{\omega}_B^\times$ belongs to the Lie algebra $so(3)$, derived from $\boldsymbol{\omega}_B$. The total thrust F and control torque $\boldsymbol{\tau} = [\tau_\phi, \tau_\theta, \tau_\psi]^\top$ given below are generated by the four rotating propellers of the quadrotor

$$\begin{bmatrix} F \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \begin{bmatrix} -c_t & -c_t & -c_t & -c_t \\ 0 & -c_t l & 0 & c_t l \\ c_t l & 0 & -c_t l & 0 \\ -c_q & c_q & -c_q & c_q \end{bmatrix} \begin{bmatrix} \Omega_1^2 \\ \Omega_2^2 \\ \Omega_3^2 \\ \Omega_4^2 \end{bmatrix}\quad (2)$$

where l denotes the quadrotor's arm length, while c_t and c_q represent the thrust coefficient and drag coefficient, respectively. The net angular speed is given by $\Omega = \Omega_1 - \Omega_2 + \Omega_3 - \Omega_4$, where Ω_i ($i = 1, 2, 3, 4$) denotes the angle speed of each of the four propellers.

Given that disturbances may originate from various unknown sources, such as aerodynamic resistance and the force exerted by payloads suspended via cables, a common approach is to model their dynamics as random walks [23]

$$\begin{aligned}\mathbf{d}_{f,k} &= \mathbf{d}_{f,k-1} + \boldsymbol{\eta}_{f,k} \\ \mathbf{d}_{\tau,k} &= \mathbf{d}_{\tau,k-1} + \boldsymbol{\eta}_{\tau,k}\end{aligned}\quad (3)$$

where $\boldsymbol{\eta}_{f,k} \sim \mathcal{N}(0, Q_f)$ and $\boldsymbol{\eta}_{\tau,k} \sim \mathcal{N}(0, Q_\tau)$ are zero-mean Gaussian noise terms, with Q_f and Q_τ as diagonal covariance matrices.

In the context of real-time dynamic optimization, the quadrotor's nonlinear dynamics are discretized using a direct multiple shooting approach with a time step of Δt . A fourth-order explicit Runge-Kutta method, applied twice within each

interval, is used for numerical integration. To accurately reflect real-world conditions, the model incorporates process noise and measurement noise, accounting for unmodeled dynamics and sensor inaccuracies. The quadrotor state is defined as $\mathbf{x} = [\mathbf{p}_E, \mathbf{v}_E, \mathbf{q}_{BE}, \boldsymbol{\omega}_B]^\top$ and the control input as $\mathbf{u} = [F, \tau_\phi, \tau_\theta, \tau_\psi]^\top$. The final dynamics model can then be written as

$$\begin{aligned}\mathbf{x}_{k+1} &= \mathbf{f}_{\text{dyn}}(\mathbf{x}_k, \mathbf{u}_k, \mathbf{d}_k, \mathbf{c}) \\ \mathbf{y}_k &= \mathbf{h}(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{v}_k.\end{aligned}\quad (4)$$

Here, the state vector $\mathbf{x} \in \mathbb{R}^{n_x}$, control input vector $\mathbf{u} \in \mathbb{R}^{n_u}$ and parameter vector $\mathbf{c} \in \mathbb{R}^{n_c}$ consist of the following components:

$$\begin{aligned}\mathbf{x} &= [x, y, z, u, v, w, q_0, q_1, q_2, q_3, p, q, r]^\top \\ \mathbf{u} &= [F, \tau_\phi, \tau_\theta, \tau_\psi]^\top \\ \mathbf{c} &= [c_t, c_q, I_{xx}, I_{yy}, I_{zz}, m]^\top\end{aligned}\quad (5)$$

and the state functions $\mathbf{f}_{\text{dyn}}$ include both the quadrotor dynamics (1) and the disturbance model (3). The measurement functions, denoted by $\mathbf{h}$, are defined without noise, while $\mathbf{y}$ represents the measurements subject to noise $\mathbf{v}$. Typically, the parameters within the vector $\mathbf{c}$ cannot be measured directly, making it necessary to employ an observer for their estimation.

III. PROBLEM FORMULATION

A. NMHE

MHE is often regarded as a dual problem of MPC, as both share a similar optimization framework. In resolving the NMHE problem, the system's state is estimated by leveraging a series of past observations, continuously refining the estimate in real time. At each time step $t = T, T+1, \dots$, NMHE optimizes over $T+1$ state vectors $\mathbf{x} = \{\mathbf{x}_k\}_{k=t-T}^t$ and T process noise vectors $\mathbf{d} = \{\mathbf{d}_k\}_{k=t-T}^{t-1}$, informed by a sequence of $T+1$ measurements $\{\mathbf{y}_k\}_{k=t-T}^t$ and control inputs $\{\mathbf{u}_k\}_{k=t-T}^{t-1}$ within a moving fixed-length horizon of length T . The estimated model parameters $\mathbf{c}$ are included in the optimization and constrained by $\mathbf{c}_{\text{max}}$. Based on the nonlinear system dynamics (4), the NMHE optimization problem can be formulated as

$$\begin{aligned}\min_{\mathbf{x}_k, \mathbf{d}_k, \mathbf{c}} \quad & \underbrace{\frac{1}{2} \sum_{k=t-T}^t \|\mathbf{y}_k - \mathbf{h}(\mathbf{x}_k, \mathbf{u}_k)\|_{\mathbf{V}}^2 + \frac{1}{2} \sum_{k=t-T}^{t-1} \|\mathbf{d}_k\|_{\mathbf{W}}^2}_{\text{running cost}} \\ & + \underbrace{\frac{1}{2} \left\| \begin{bmatrix} \mathbf{x}_{t-T} - \hat{\mathbf{x}}_{t-T} \\ \mathbf{c} - \hat{\mathbf{c}}_{t-T} \end{bmatrix} \right\|_{\mathbf{P}}^2}_{\text{arrival cost}} \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = \mathbf{f}_{\text{dyn}}(\mathbf{x}_k, \mathbf{u}_k, \mathbf{d}_k, \mathbf{c}), \quad k = t-T, \dots, t-1 \\ & \|\mathbf{x}_k\| \leq \mathbf{x}_{k, \text{max}}, \quad k = t-T, \dots, t \\ & \|\mathbf{c}\| \leq \mathbf{c}_{\text{max}}\end{aligned}\quad (6)$$

where t represents the current time, while k indexes the estimation horizon window. The running cost penalizes deviations in measurement and process noise, while the arrival cost incorporates prior estimates [24]. The NMHE result $\hat{\mathbf{x}}_{t-T}$ represents

the prior estimate of $\mathbf{x}_{t-T}$ computed at time $t-1$. Similarly, $\hat{\mathbf{c}}_{t-T}$ denotes the parameter estimate at the start of the estimation horizon. The weight matrices $\mathbf{V}$, $\mathbf{W}$, and $\mathbf{P}$ are defined based on the covariance matrices of measurement noise, process noise, and initial state uncertainties, respectively [25]

$$\mathbf{V} = \mathbf{Q}^{-1/2}, \quad \mathbf{W} = \mathbf{R}^{-1/2}, \quad \mathbf{P} = \mathbf{Q}_0^{-1/2}. \quad (7)$$

The performance of NMHE, particularly its learning rate, depends on the choice of estimation horizon T , which is largely problem-specific. A larger T improves accuracy but increases computational demand, whereas a smaller T enhances responsiveness at the expense of estimation quality. The initial states and parameters are initialized using prior knowledge and refined during simulations, ensuring robust state estimation in dynamic environments.

B. NMPC

NMPC is an advanced, optimization-based approach to feedback control. It determines the optimal control action by solving a constrained, finite-horizon optimal control problem (OCP) at each time step. This real-time open-loop optimization, performed over a prediction horizon N , generates a sequence of control inputs, of which only the first is applied as the current control action [26].

To accommodate various tracking scenarios, we structure the discrete-time parametric OCP for NMPC as a least squares objective. This objective minimizes the deviations of the state $\mathbf{x}_k$ and control inputs $\mathbf{u}_k$ from their respective targets over the defined prediction horizon ($t \leq k \leq t+N$)

$$\begin{aligned}\min_{\mathbf{x}, \mathbf{u}} \quad & \frac{1}{2} \sum_{k=t}^{t+N-1} \left(\|\mathbf{x}'_k - \mathbf{x}_k^{\text{ref}}\|_{\mathbf{W}_x}^2 + \|\mathbf{u}_k - \mathbf{u}_k^{\text{ref}}\|_{\mathbf{W}_u}^2 \right) \\ & + \frac{1}{2} \|\mathbf{x}'_N - \mathbf{x}_N^{\text{ref}}\|_{\mathbf{W}_N}^2 \\ \text{s.t.} \quad & \mathbf{x}'_t = \hat{\mathbf{x}}_t, \\ & \mathbf{x}'_{k+1} = \mathbf{f}_{\text{dyn}}(\mathbf{x}'_k, \mathbf{u}_k, \mathbf{c}), \quad k = t, \dots, t+N-1 \\ & \mathbf{H}_x \mathbf{x}'_k \leq \mathbf{h}_x, \quad k = t, \dots, t+N \\ & \mathbf{H}_u \mathbf{u}_k \leq \mathbf{h}_u, \quad k = t, \dots, t+N-1\end{aligned}\quad (8)$$

where $\mathbf{x}'_k \in \mathbb{R}^{n_x}$ represents the vector of predicted state, $\mathbf{u}_k \in \mathbb{R}^{n_u}$ denotes the control input, and the NMHE result $\hat{\mathbf{x}}_t \in \mathbb{R}^{n_x}$ serves as the initial condition for the NMPC optimization, linking the estimation (NMHE) and control processes (NMPC). The time-varying references for the state and control inputs are indicated by $\mathbf{x}_k^{\text{ref}}$ and $\mathbf{u}_k^{\text{ref}}$, respectively, while the terminal state reference is given by $\mathbf{x}_N^{\text{ref}}$. The weight matrices for the state, control input, and terminal state, denoted as $\mathbf{W}_x \in \mathbb{R}^{n_x \times n_x}$, $\mathbf{W}_u \in \mathbb{R}^{n_u \times n_u}$, and $\mathbf{W}_N \in \mathbb{R}^{n_x \times n_x}$, respectively, are assumed to be constant throughout the analysis. Additionally, the state and control input constraints are defined by their respective maximum bounds, $\|\mathbf{x}'_k\| \leq \mathbf{x}_{k, \text{max}} \in \mathbb{R}^{n_x}$ and $\|\mathbf{u}_k\| \leq \mathbf{u}_{k, \text{max}} \in \mathbb{R}^{n_u}$.

Nevertheless, the efficacy of NMPC, in both the practicality of anticipated trajectories and tracking precision, heavily relies on the careful tuning of various hyperparameters,

particularly the weight matrices. Specifically, we define $\theta = \{\mathbf{W}_x, \mathbf{W}_u, \mathbf{W}_N\}$ as the hyperparameters for NMPC. Consequently, we can represent problem (8) as $\Xi(\theta)$ and denote the predicted trajectories of NMPC as $\zeta(\theta) = \{\mathbf{x}'(\theta), \mathbf{u}(\theta)\}$, respectively.

C. Closed-Loop System Sensitivity

Closed-loop training allows for greater flexibility compared to open-loop training, as its loss function is not constrained by the NMPC's prediction horizon N . This flexibility is particularly advantageous for learning complex tasks and optimizing control strategies. Unlike open-loop training, which often depends on expert trajectories that are difficult to acquire, closed-loop training leverages RL to interact directly with the environment. This approach evaluates system performance through interaction rather than imitation learning [27], making it more adaptable to various scenarios.

To learn the NMPC controller $\Xi(\theta)$, which functions as a feedback controller tracking a desired state $\mathbf{x}_k^{\text{ref}}$ by generating the control law $\mathbf{u}_k = \mathcal{U}(\mathbf{x}'_k, \mathbf{x}_k^{\text{ref}}, \theta)$, we can formulate the following parameterized optimization problem:

$$\begin{aligned} \min_{\theta} \quad & L(\mathbf{x}', \mathbf{x}^{\text{ref}}, \mathbf{u}; \theta) \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = \mathbf{f}_{\text{dyn}}(\mathbf{x}'_k, \mathbf{u}_k, \mathbf{c}), \mathbf{x}'_t = \hat{\mathbf{x}}_t \text{ is estimated by NMHE} \\ & \mathbf{u}_k \text{ are generated by } \Xi(\theta), k \in \{t, t+1, \dots, t+N-1\}. \end{aligned} \quad (9)$$

Here, the objective is to find the optimal hyperparameters θ^* such that the NMPC predicted trajectories $\zeta^*(\theta)$ minimize the loss function $L(\cdot)$, which is based on the closed-loop trajectory tracking errors over a horizon of length N_c .

We aim to solve problem (9) to enable the auto-tuning of NMPC within an end-to-end learning framework. The gradient of the loss L with respect to the hyperparameters θ is obtained by applying the chain rule [28]

$$\nabla_{\theta} L = \left(\frac{\partial \mathbf{x}^*}{\partial \theta} \right)^{\top} \nabla_{\mathbf{x}} L(\mathbf{x}^*) + \left(\frac{\partial \mathbf{u}^*}{\partial \theta} \right)^{\top} \nabla_{\mathbf{u}} L(\mathbf{u}^*) \quad (10)$$

where the state $\mathbf{x}^*$ and control input $\mathbf{u}^*$ are the optimal solutions to problem (8).

The sensitivity propagation can be defined as follows:

$$\begin{aligned} \frac{\partial \mathbf{x}_{t+1}}{\partial \theta} &= \left(\frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \mathbf{x}_t} + \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \mathbf{u}_t} \frac{\partial \mathbf{u}}{\partial \theta} \right) \frac{\partial \mathbf{x}_t}{\partial \theta} + \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \mathbf{u}_t} \frac{\partial \mathbf{u}}{\partial \theta} \\ \frac{\partial \mathbf{u}_t}{\partial \theta} &= \frac{\partial \mathbf{u}}{\partial \mathbf{x}_t} \frac{\partial \mathbf{x}_t}{\partial \theta} + \frac{\partial \mathbf{u}}{\partial \theta_t} \end{aligned} \quad (11)$$

where the gradients $\partial \mathbf{f}_{\text{dyn}} / \partial \mathbf{x}_t$ and $\partial \mathbf{f}_{\text{dyn}} / \partial \mathbf{u}_t$ are straightforward to compute as they derive directly from the system dynamics. The main challenge lies in calculating $\partial \mathbf{u} / \partial \mathbf{x}_t$ and $\partial \mathbf{u}_t / \partial \theta$, which capture the sensitivity of the controller to the state $\mathbf{x}_t$ and the hyperparameters θ . These sensitivities are essential for linking the hyperparameters to the optimal trajectory and are computed via sensitivity propagation, ensuring the differentiability of the NMPC framework.

IV. LEARNING-BASED NMHE-DNMPC FRAMEWORK

A. Transformation of Quadrotor Dynamics

To find the optimal trajectory $\zeta^*(\theta) = \{\mathbf{x}^*(\theta), \mathbf{u}^*(\theta)\}$, the problem $\Xi(\theta)$ is reformulated into a sequential quadratic programming (SQP) problem subject to both equality and inequality constraints [29], utilizing the first-order Taylor series expansion of the dynamics (4) around the optimal solution trajectory $\{\tilde{\mathbf{x}}_t, \tilde{\mathbf{u}}_t\}_{t=1:N}$

$$\mathbf{x}_{t+1} = \mathbf{A}_t \mathbf{x}_t + \mathbf{B}_t \mathbf{u}_t + \mathbf{n}_t \quad (12)$$

where

$$\begin{aligned} \mathbf{A}_t &= \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \mathbf{x}} \bigg|_{\tilde{\mathbf{x}}, \tilde{\mathbf{u}}} = \begin{bmatrix} \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \mathbf{p}_E}, \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \mathbf{v}_E}, \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \mathbf{q}_{BE}}, \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \boldsymbol{\omega}_B} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\partial}{\partial \mathbf{p}_E} \dot{\mathbf{p}}_E & \frac{\partial}{\partial \mathbf{v}_E} \dot{\mathbf{p}}_E & \frac{\partial}{\partial \mathbf{q}_{BE}} \dot{\mathbf{p}}_E & \frac{\partial}{\partial \boldsymbol{\omega}_B} \dot{\mathbf{p}}_E \\ \frac{\partial}{\partial \mathbf{p}_E} \dot{\mathbf{v}}_E & \frac{\partial}{\partial \mathbf{v}_E} \dot{\mathbf{v}}_E & \frac{\partial}{\partial \mathbf{q}_{BE}} \dot{\mathbf{v}}_E & \frac{\partial}{\partial \boldsymbol{\omega}_B} \dot{\mathbf{v}}_E \\ \frac{\partial}{\partial \mathbf{p}_E} \dot{\mathbf{q}}_{BE} & \frac{\partial}{\partial \mathbf{v}_E} \dot{\mathbf{q}}_{BE} & \frac{\partial}{\partial \mathbf{q}_{BE}} \dot{\mathbf{q}}_{BE} & \frac{\partial}{\partial \boldsymbol{\omega}_B} \dot{\mathbf{q}}_{BE} \\ \frac{\partial}{\partial \mathbf{p}_E} \dot{\boldsymbol{\omega}}_B & \frac{\partial}{\partial \mathbf{v}_E} \dot{\boldsymbol{\omega}}_B & \frac{\partial}{\partial \mathbf{q}_{BE}} \dot{\boldsymbol{\omega}}_B & \frac{\partial}{\partial \boldsymbol{\omega}_B} \dot{\boldsymbol{\omega}}_B \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{0} & \mathbf{I}_{3 \times 3} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & 2m^{-1} \mathbf{F} \mathbf{R}_{BE} \mathbf{Q} \mathbf{e}_3 & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \frac{1}{2} \begin{bmatrix} \mathbf{0} & -\mathbf{I}_{3 \times 3} \\ \mathbf{I}_{3 \times 3} & -\boldsymbol{\omega}^{\times} \end{bmatrix} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{J}_B^{-1} (-\boldsymbol{\omega}^{\times} \mathbf{J}_B - \mathbf{J}_B \boldsymbol{\omega}^{\times}) \end{bmatrix} \\ \mathbf{B}_t &= \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \mathbf{u}} \bigg|_{\tilde{\mathbf{x}}, \tilde{\mathbf{u}}} = \begin{bmatrix} \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial F}, \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \tau_{\phi}}, \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \tau_{\theta}}, \frac{\partial \mathbf{f}_{\text{dyn}}}{\partial \tau_{\psi}} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\partial}{\partial F} \dot{\mathbf{p}}_E & \frac{\partial}{\partial \tau_{\phi}} \dot{\mathbf{p}}_E & \frac{\partial}{\partial \tau_{\theta}} \dot{\mathbf{p}}_E & \frac{\partial}{\partial \tau_{\psi}} \dot{\mathbf{p}}_E \\ \frac{\partial}{\partial F} \dot{\mathbf{v}}_E & \frac{\partial}{\partial \tau_{\phi}} \dot{\mathbf{v}}_E & \frac{\partial}{\partial \tau_{\theta}} \dot{\mathbf{v}}_E & \frac{\partial}{\partial \tau_{\psi}} \dot{\mathbf{v}}_E \\ \frac{\partial}{\partial F} \dot{\mathbf{q}}_{BE} & \frac{\partial}{\partial \tau_{\phi}} \dot{\mathbf{q}}_{BE} & \frac{\partial}{\partial \tau_{\theta}} \dot{\mathbf{q}}_{BE} & \frac{\partial}{\partial \tau_{\psi}} \dot{\mathbf{q}}_{BE} \\ \frac{\partial}{\partial F} \dot{\boldsymbol{\omega}}_B & \frac{\partial}{\partial \tau_{\phi}} \dot{\boldsymbol{\omega}}_B & \frac{\partial}{\partial \tau_{\theta}} \dot{\boldsymbol{\omega}}_B & \frac{\partial}{\partial \tau_{\psi}} \dot{\boldsymbol{\omega}}_B \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ m^{-1} \mathbf{R}_{BE} \mathbf{e}_3 & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{J}_B^{-1} \mathbf{e}_1 & \mathbf{J}_B^{-1} \mathbf{e}_2 & \mathbf{J}_B^{-1} \mathbf{e}_3 \end{bmatrix} \end{aligned}$$

$$\mathbf{n}_t = \mathbf{f}_{\text{dyn}}(\tilde{\mathbf{x}}, \tilde{\mathbf{u}}) - \mathbf{A}_t \tilde{\mathbf{x}} - \mathbf{B}_t \tilde{\mathbf{u}}$$

where $\mathbf{e}_1 = [1, 0, 0]^{\top}$, $\mathbf{e}_2 = [0, 1, 0]^{\top}$, $\mathbf{e}_3 = [0, 0, 1]^{\top}$, $\mathbf{Q} = \begin{bmatrix} -q_2 & q_3 & q_0 & -q_1 \\ -q_3 & -q_2 & q_1 & -q_0 \\ q_2 & q_1 & -q_0 & q_3 \end{bmatrix}$. Therefore, we can obtain the following equation:

$$\mathbf{F}_{xt} \mathbf{x}_t + \mathbf{F}_{ut} \mathbf{u}_t = \mathbf{f}_t \quad (13)$$

$$\mathbf{G}_x \mathbf{x}_t \leq \mathbf{g}_x, \mathbf{G}_u \mathbf{u}_t \leq \mathbf{g}_u \quad (14)$$

$$\text{where } \mathbf{F}_{xt} = \begin{bmatrix} \mathbf{I} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{A}_t & -\mathbf{I} & \mathbf{0} & \cdots & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{A}_t & -\mathbf{I} & \cdots & \mathbf{0} & \mathbf{0} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{A}_t & -\mathbf{I} \end{bmatrix}, \mathbf{f}_t = \begin{bmatrix} \tilde{\mathbf{x}}_t \\ -\mathbf{n}_t \\ -\mathbf{n}_t \\ \vdots \\ -\mathbf{n}_t \end{bmatrix},$$

$$F_{ut} = \begin{bmatrix} \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ B_t & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & B_t & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & B_t \end{bmatrix}, \quad g_x = \begin{bmatrix} h_x \\ h_x \\ \vdots \\ h_{x,N} \end{bmatrix}, \quad g_u = \begin{bmatrix} h_u \\ h_u \\ \vdots \\ h_u \end{bmatrix},$$

$$G_x = \begin{bmatrix} H_x & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & H_x & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & H_{x,N} \end{bmatrix}, \quad G_u = \begin{bmatrix} H_u & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & H_u & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & H_u \end{bmatrix}.$$

Then, the optimization variable $\mathbf{z}$ is defined as $\mathbf{z} = (\mathbf{u}(t), \mathbf{x}(t+1), \dots, \mathbf{u}(t+N-1), \mathbf{x}(t+N)) \in \mathbb{R}^{N(n_x+n_u)}$ to determine the optimal solution $\mathbf{z}^*$. Then, $\Xi(\theta)$ can be expressed as

$$\begin{aligned} \min \quad & \frac{1}{2} \mathbf{z}^\top \mathbf{H} \mathbf{z} + \mathbf{q}^\top \mathbf{z} \\ \text{s.t.} \quad & \mathbf{F}_t \mathbf{z} = \mathbf{f}_t, \mathbf{G} \mathbf{z} \leq \mathbf{g} \end{aligned} \quad (15)$$

where $\mathbf{H} = \text{diag}(\mathbf{W}_x, \dots, \mathbf{W}_x, \mathbf{W}_N, \mathbf{W}_u, \dots, \mathbf{W}_u)$, $\mathbf{F}_t = [\mathbf{F}_{xt}, \mathbf{F}_{ut}]$, $\mathbf{G} = \text{diag}(\mathbf{G}_x, \mathbf{G}_u)$, $\mathbf{g} = [\mathbf{g}_x, \mathbf{g}_u]^\top$.

B. Differentiating NMPC

Building on the sensitivity analysis discussed in Section III-C, we now define the Lagrangian $\mathcal{L}$ for backward propagation, which forms the foundation for computing the gradients of the loss function with respect to the hyperparameters. The Lagrangian is formulated as follows:

$$\mathcal{L}(\mathbf{z}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = \frac{1}{2} \mathbf{z}^\top \mathbf{H} \mathbf{z} + \mathbf{q}^\top \mathbf{z} + \boldsymbol{\mu}^\top (\mathbf{F}_t \mathbf{z} - \mathbf{f}_t) + \boldsymbol{\lambda}^\top (\mathbf{G} \mathbf{z} - \mathbf{g}) \quad (16)$$

where $\boldsymbol{\mu}$ and $\boldsymbol{\lambda} \geq 0$ are the dual variables corresponding to the equality and inequality constraints, respectively. Differentiating the Lagrangian $\mathcal{L}$ with respect to the optimization variable $\mathbf{z}$, and the dual variables $\boldsymbol{\mu}$ and $\boldsymbol{\lambda}$ yields the following functions:

$$\begin{aligned} \mathbf{H} \mathbf{z} + \mathbf{q} + \mathbf{F}^\top \boldsymbol{\mu} + \mathbf{G}^\top \boldsymbol{\lambda} &= \mathbf{0} \\ \mathbf{F} \mathbf{z} - \mathbf{f} &= \mathbf{0} \\ \text{diag}(\boldsymbol{\lambda}) (\mathbf{G} \mathbf{z} - \mathbf{g}) &= \mathbf{0}. \end{aligned} \quad (17)$$

The subsequent equalities hold after deriving the Karush–Kuhn–Tucker (KKT) conditions with respect to differential variations $d\mathbf{H}$, $d\mathbf{q}$, $d\mathbf{G}$, $d\mathbf{g}$, $d\mathbf{F}$, and $d\mathbf{f}$, at the optimal points $\mathbf{z}^*$, $\boldsymbol{\mu}^*$, $\boldsymbol{\lambda}^*$

$$\begin{aligned} d\mathbf{H} \mathbf{z}^* + \mathbf{H} d\mathbf{z} + d\mathbf{q} + (d\mathbf{F})^\top \boldsymbol{\mu}^* + \mathbf{F}^\top d\boldsymbol{\mu} \\ + (d\mathbf{G})^\top \boldsymbol{\lambda}^* + \mathbf{G}^\top d\boldsymbol{\lambda} &= \mathbf{0} \\ d\mathbf{F} \mathbf{z}^* + \mathbf{F} d\mathbf{z} - d\mathbf{f} &= \mathbf{0} \\ \text{diag}(\mathbf{G} \mathbf{z}^* - \mathbf{g}) d\boldsymbol{\lambda} + \text{diag}(\boldsymbol{\lambda}^*) (d\mathbf{G} \mathbf{z}^* + \mathbf{G} d\mathbf{z} - d\mathbf{g}) &= \mathbf{0} \end{aligned} \quad (18)$$

which can be converted into the following expression:

$$\begin{aligned} \begin{bmatrix} \mathbf{H} & \mathbf{G}^\top & \mathbf{F}^\top \\ \text{diag}(\boldsymbol{\lambda}^*) \mathbf{G} & \text{diag}(\mathbf{G} \mathbf{z}^* - \mathbf{g}) & \mathbf{0} \\ \mathbf{F} & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} d\mathbf{z} \\ d\boldsymbol{\lambda} \\ d\boldsymbol{\mu} \end{bmatrix} \\ = - \begin{bmatrix} d\mathbf{H} \mathbf{z}^* + d\mathbf{q} + \mathbf{G}^\top d\boldsymbol{\lambda} + (d\mathbf{F})^\top \boldsymbol{\mu}^* \\ \text{diag}(\boldsymbol{\lambda}^*) d\mathbf{G} \mathbf{z}^* - \text{diag}(\boldsymbol{\lambda}^*) d\mathbf{g} \\ d\mathbf{F} \mathbf{z}^* - d\mathbf{f} \end{bmatrix}. \end{aligned} \quad (19)$$

For the purpose of propagating errors backward through the Jacobian matrix $\partial L / \partial \mathbf{D}$ of a loss function L with respect to the SQP parameters $\mathbf{D} = \{\mathbf{H}, \mathbf{q}\}$, we utilize the gradient vector $\partial L / \partial \mathbf{z}^*$ obtained from the preceding backward pass. This gradient vector is then scaled by the derivative $\partial \mathbf{z}^* / \partial \mathbf{D}$ according to the chain rule of calculus, resulting in the expression $(\partial L / \partial \mathbf{D}) = (\partial L / \partial \mathbf{z}^*) (\partial \mathbf{z}^* / \partial \mathbf{D})$, then

$$\begin{bmatrix} d\mathbf{z} \\ d\boldsymbol{\lambda} \\ d\boldsymbol{\mu} \end{bmatrix} = - \begin{bmatrix} \mathbf{H} & \mathbf{P}^\top \text{diag}(\boldsymbol{\lambda}^*) & \mathbf{F}^\top \\ \mathbf{G} & \text{diag}(\mathbf{G} \mathbf{z}^* - \mathbf{g}) & \mathbf{0} \\ \mathbf{C} & \mathbf{0} & \mathbf{0} \end{bmatrix} \begin{bmatrix} (\frac{\partial L}{\partial \mathbf{z}^*})^\top \\ \mathbf{0} \\ \mathbf{0} \end{bmatrix}. \quad (20)$$

The gradients of the loss function L with respect to $\mathbf{D}$, are computed using the following set of formulas. For additional details, can refer to [30]

$$\nabla_{\mathbf{H}} L = \frac{1}{2} (d\mathbf{z} \mathbf{z}^\top + \mathbf{z} d\mathbf{z}^\top), \quad \nabla_{\mathbf{q}} L = d\mathbf{z}. \quad (21)$$

To update the parameters $\mathbf{D}$, we employ the projected stochastic gradient method [31]. At each learning epoch $t \in \mathbb{N}$, the loss function is evaluated, and its gradient with respect to the current parameter estimate, $\nabla_{\mathbf{D}} L$, is calculated. Differentiating through the KKT conditions to compute $\nabla_{\mathbf{D}} L$ can lead to numerical instabilities due to the high-dimensional and nonlinear nature of the problem. To address this, diagonal regularization is applied to ensure the positive definiteness of the Hessian, while automatic differentiation (AD) is leveraged for accurate and efficient gradient computation. The parameters are then updated as

$$\hat{\mathbf{D}}_{t+1} = \mathcal{P}(\hat{\mathbf{D}}_t - \alpha_t \nabla_{\mathbf{D}} L) \quad (22)$$

where $\mathcal{P}$ is the projection onto the feasible set $\mathcal{S}$, and α_t is a learning rate satisfies the following conditions to ensure convergence:

$$\sum_{t=0}^{\infty} \alpha_t^2 < \infty, \quad \sum_{t=0}^{\infty} \alpha_t = \infty. \quad (23)$$

A learning rate that achieves good practical convergence for unconstrained stochastic gradient descent is given by [32]

$$\alpha_t = \frac{\alpha_0}{t}, \quad \forall t \geq 1 \quad (24)$$

with $\alpha_0 > 0$. This choice balances convergence speed and stability in stochastic optimization problems.

C. Learning-Based Cascade Framework Design

We design a learning-based cascade framework for implementing DNMPC on quadrotors with real-time online estimation. This framework integrates NMHE to observe the states and estimate both disturbances and essential parameters of the quadrotor. These estimates are then utilized to develop a DNMPC, whose hyperparameters θ are trained end-to-end in a closed-loop fashion. The complete procedure is summarized in Algorithm 1. By integrating NMHE for efficient state estimation and employing stochastic optimization for hyperparameter updates, the framework effectively addresses real-time computational challenges. Moreover, the use of a diminishing learning

Algorithm 1: Learning-Based Cascade Framework.

Input: Previously estimated initial state $\hat{x}_{t-L}$, output measurements $\mathbf{y}$ and the pre-planned references $\mathbf{y}^{ref}$

Parameters: MHE parameters $\{L, V, W, P\}$, MPC parameters $\{N, R, Q, Q_N\}$, Choose QP parameters $\mathbf{D} = \{\mathbf{H}, \mathbf{q}\}$.

- 1: **while** the stopping criterion is not satisfied **do**
- 2: Update NMHE
- 3: Measure the output $\mathbf{y}_t$ from the system dynamics (4) at time t
- 4: Obtain $\hat{x}_t$, $\hat{d}_t$ and $\hat{c}$ from NMHE (6)
- 5: Calculate DNMP
- 6: Use the NMHE estimate $\hat{x}_t$ as the current state $\mathbf{x}'_t$
- 7: Find optimal control actions $\mathbf{u}_t^*$ by solving NMPC (8)
- 8: Update the loss L with the squared error and regularization using (25)
- 9: Define KKT conditions derived from the Lagrangian $\mathcal{L}$ in (16)
- 10: Differentiate the KKT conditions with respect to admissible variations
- 11: Obtain the gradient of the loss $\nabla_{\mathbf{D}} L$ using AD in (21)
- 12: Update parameters $\hat{\mathbf{D}}_{t+1}$ using gradient-based methods from (22)
- 13: **end while**

rate α_t and a projection operator $\mathcal{P}$ ensures stability and convergence, even with nonlinear and nonconvex loss functions.

Our objective is to train θ by minimizing the loss function L , designed to evaluate tracking performance. Specifically, the loss $L(\mathbf{x}, \mathbf{x}^{ref}, \mathbf{u}; \theta)$ is constructed from the closed-loop tracking errors accumulated over the horizon N_c , defined as

$$L(\mathbf{x}, \mathbf{x}^{ref}, \mathbf{u}; \theta) = (\mathbf{y}^{ref}(0) - \mathbf{y}(0))^2 + \sum_{i=1}^{N_c} (\mathbf{y}^{ref}(i) - \mathbf{y}(\hat{\mathbf{x}}(\mathbf{u}(i))))^2 \quad (25)$$

where $\mathbf{y}_{ref}(i)$ represents the time-sampled reference, and $\mathbf{y}(\hat{\mathbf{x}}(\mathbf{u}(i)))$ denotes the estimated state over the time horizon N_c that requires minimization.

V. COMPARISON TESTS AND STUDIES

To validate the presented learning-based NMHE-DNMP framework for trajectory tracking of a quadrotor with 6 Degrees of Freedom (DoF) nonlinear dynamics, we implemented it in a simulation environment using a Crazyflie quadrotor UAV, as illustrated in Fig. 3 [33]. The performance of the framework was subsequently evaluated for trajectory tracking under the influence of external disturbances, which are modeled as bounded, time-varying external forces that impact the system's state. Specifically, the external disturbance was uniformly sampled from the range $[-1, 1]$ for both linear and angular velocities. The quadrotor starts its maneuver from the initial position $\mathbf{p}_0 = [0, 0, 0]^T$ (m). The physical parameters of the quadrotor model (1), along with the estimation gains and the prescribed performance parameters

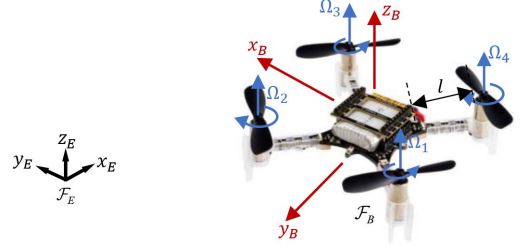


Fig. 3. Crazyflie quadrotor UAV.

TABLE I
CRAZYFLIE QUADROTOR INTRINSIC PARAMETERS

Description	Value
Mass of quadrotor m	0.054 kg
Moment of inertia about I_{xx}	$1.6571710 \times 10^{-4} \text{ kgm}^2$
Moment of inertia about I_{yy}	$1.6655602 \times 10^{-4} \text{ kgm}^2$
Moment of inertia about I_{zz}	$2.9261652 \times 10^{-4} \text{ kgm}^2$
Aerodynamic thrust coefficient c_t	1 Ns ²
Aerodynamic drag coefficient c_d	0.005964552 Nms ²
Arm length l	0.367 m

used in the simulations, are detailed in Table I. Here, m , I_{xx} , I_{yy} , and I_{zz} represent the actual mass and moments of inertia of the quadrotor, which are assumed to be unknown, while $m_0 = 0.027 \text{ kg}$, $I_{xx0} = 1.6571710 \times 10^{-5} \text{ kgm}^2$, $I_{yy0} = 1.6655602 \times 10^{-5} \text{ kgm}^2$, and $I_{zz0} = 2.9261652 \times 10^{-5} \text{ kgm}^2$ denote the corresponding known initial values.

For entire experiments, we utilized an interpretable quadratic cost function, parameterized by θ , within the NMPC formulation. The loss function L represents the sum of squared tracking errors of the closed-loop system compared to the reference trajectory. The primary problem is addressed using ACADOS [34] within a Python 3.7 environment. The experiments are conducted on a computer with a 2.9 GHz Intel Core i5 processor and 16 GB of RAM. Table I shows the quadrotor's parameters.

Here, we consider two reference trajectories: one is a circle trajectory, and the other is a 3-D lemniscate trajectory. For both cases, the NMHE problem is configured with an estimation horizon of 0.5 s and a sample time of 0.05 s ($T = 10$), using the state, control, and parameter vectors specified in (5). Additionally, the weight matrices are set as follows:

$$\begin{aligned} \mathbf{P} &= \text{diag}(0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, \\ &\quad 0.1, 0.032) \\ \mathbf{W} &= \text{diag}(0.1, 0.1, 0.14, 0.14, 0.14, 0.14, 0.14, 0.14, 0.14, \\ &\quad 0.17, 0.17, 0.17, 0.17, 0.17, 0.89) \\ \mathbf{V} &= \text{diag}(0.032, 0.032, 0.032, 0.045, 0.045, 0.045, 0.022, \\ &\quad 0.022, 0.022, 0.224, 0.224, 0.224, 0.003, 0.003, 0.003). \end{aligned}$$

Besides, the arrival cost is initialized with the state and parameter vectors $\bar{\mathbf{x}}_L = [0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]^T$ and $\bar{\mathbf{c}} = [2, 5 \times 10^{-3}, I_{xx0}, I_{yy0}, I_{zz0}, m_0]^T$. In the NMPC problem, the prediction horizon length N is set to 20, and $\mathbf{W}_x$, $\mathbf{W}_N$ and $\mathbf{W}_u$ are initialized as I_{13} , I_{13} , and I_4 , respectively.

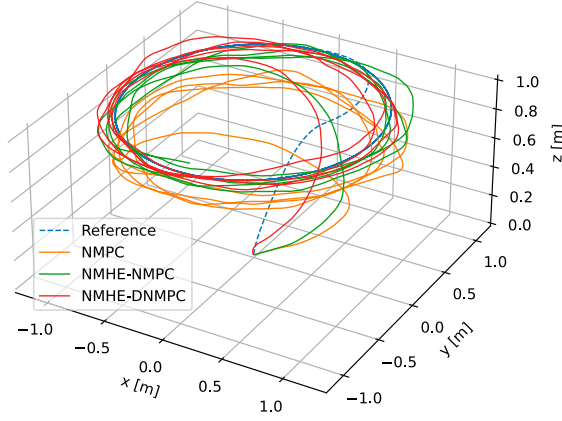


Fig. 4. Tracking trajectories of the quadrotor for the circle trajectory.

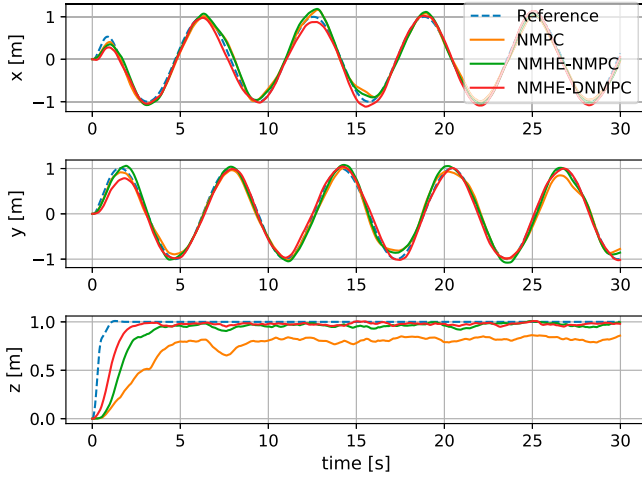


Fig. 5. Position tracking performance of the quadrotor for the circle trajectory.

Both simulations, using the NMHE-DNMPC approach, were trained over 20 epochs.

The tracking performance of the learning-based NMHE-DNMPC for the circle-shaped trajectory can be visualized in Fig. 4. As shown in Fig. 5, which presents the position tracking performance, and Fig. 6, which shows the evolution of the position error, the proposed approach demonstrates superior performance. Specifically, the RMSE errors for position tracking in the circle-shaped reference using NMPC, NMHE-NMPC, and NMHE-DNMPC methods are 0.3235, 0.2074, and 0.1603 m, respectively.

Fig. 7 shows the performance of the quadrotor tracking the lemniscate trajectory, which has a more complex shape and imposes higher requirements on the control system, especially at the corners and intersections of the trajectories. The position tracking performance of the quadrotor using the proposed approach, as shown in Fig. 8, is better than that of the quadrotor using NMPC and NMHE-NMPC. Due to the learning process, the quadrotor exhibits a smaller tracking error of 0.3365 m, compared to 0.3464 m for NMHE-NMPC and 8.0381 m for NMPC, as shown in Fig. 9.

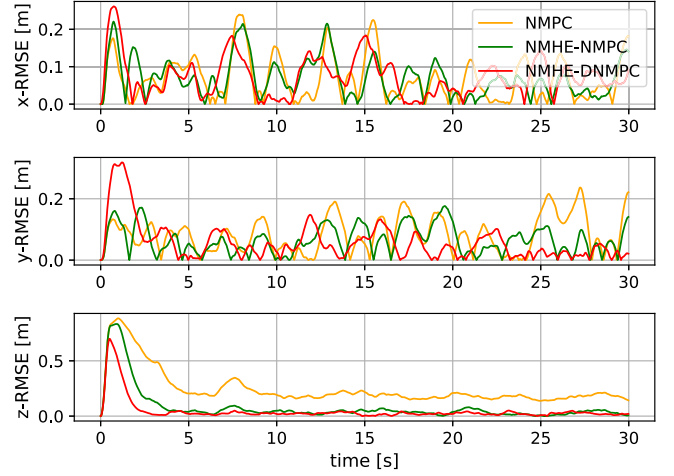


Fig. 6. Position tracking error of the quadrotor for the circular trajectory.

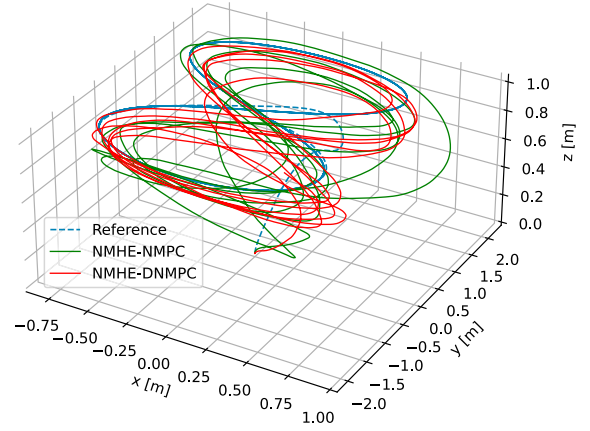


Fig. 7. Tracking trajectories of the quadrotor for the lemniscate trajectory.

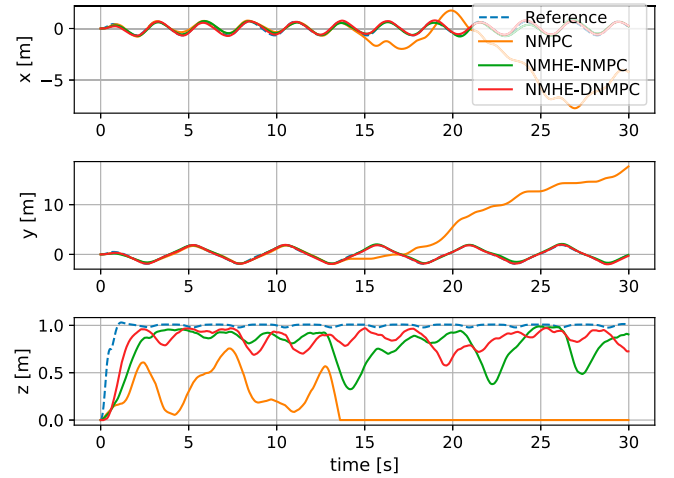


Fig. 8. Position tracking performance of the quadrotor for the lemniscate trajectory.

We further evaluate the computational efficiency of the proposed NMHE-DNMPC framework by measuring the running time required for the entire process per iteration. Table II

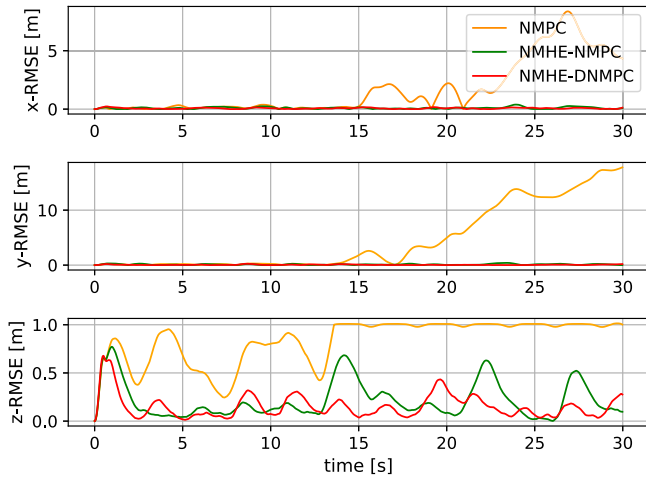


Fig. 9. Position tracking error of the quadrotor for the lemniscate trajectory.

TABLE II
AVERAGE RUNNING TIME PER ITERATION FOR
NMHE-DNMPC ACROSS TRAJECTORIES

Trajectory Type	NMHE Time (s)	DNMPC Time (s)	Total Time (s)
Circle-shaped	0.004	0.027	0.031
Lemniscate	0.004	0.03	0.034

summarizes the average running time across different trajectory references. These results reflect the computational feasibility of implementing the proposed framework on real-world platforms.

VI. CONCLUSION

A learning-based NMHE-DNMPC approach was designed for quadrotor trajectory tracking. The NMHE estimator was employed to simultaneously estimate the system states, external disturbances, and key parameters, feeding this information into the DNMPC for improved control. By incorporating the DiffTune method, the controller's hyperparameters were automatically tuned in an end-to-end manner within a closed-loop framework, enabling the system to adapt to time-varying environments. Comparative studies demonstrated the proposed method's effectiveness and superiority, validating its robustness and adaptability in dynamic conditions.

In the future, we will extend our approach to handle more complex disturbances and incorporate real-time system identification to enhance adaptability in dynamic environments. Additionally, exploring the application of this framework to multiagent systems could improve coordination and scalability for various tasks.

REFERENCES

- [1] K. Telli et al., "A comprehensive review of recent research trends on unmanned aerial vehicles (UAVs)," *Systems*, vol. 11, no. 8, p. 400, 2023.
- [2] S. Belkhal, R. Li, G. Kahn, R. McAllister, R. Calandra, and S. Levine, "Model-based meta-reinforcement learning for flight with suspended payloads," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 1471–1478, Apr. 2021.
- [3] L. Xi, X. Wang, L. Jiao, S. Lai, Z. Peng, and B. M. Chen, "GTO-MPC-based target chasing using a quadrotor in cluttered environments," *IEEE Trans. Ind. Electron.*, vol. 69, no. 6, pp. 6026–6035, Jun. 2022.
- [4] F. Gonzalez, A. Heckmann, S. Notter, M. Zurn, J. Trachte, and A. McFadyen, "Non-linear model predictive control for UAVs with slung/swung load," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2015, pp. 1–8.
- [5] B. Lindqvist, S. S. Mansouri, A.-a. Agha-mohammadi, and G. Nikolakopoulos, "Nonlinear MPC for collision avoidance and control of UAVs with dynamic obstacles," *IEEE Robot. Automat. Lett.*, vol. 5, no. 4, pp. 6001–6008, Oct. 2020.
- [6] K. Guo, P. Tang, H. Wang, D. Lin, and X. Cui, "Autonomous landing of a quadrotor on a moving platform via model predictive control," *Aerospace*, vol. 9, no. 1, p. 34, 2022.
- [7] D. Chen, F. Hu, G. Nian, and T. Yang, "Deep residual learning for nonlinear regression," *Entropy*, vol. 22, no. 2, p. 193, 2020.
- [8] X. Liu, Z. Yuan, Z. Gao, and W. Zhang, "Reinforcement learning-based fault-tolerant control for quadrotor UAVs under actuator fault," *IEEE Trans. Ind. Inform.*, vol. 20, no. 12, pp. 13926–13935, Dec. 2024.
- [9] T. Salzmann, E. Kaufmann, J. Arrizabalaga, M. Pavone, D. Scaramuzza, and M. Ryll, "Real-time neural MPC: Deep learning model predictive control for quadrotors and agile robotic platforms," *IEEE Robot. Automat. Lett.*, vol. 8, no. 4, pp. 2397–2404, Apr. 2023.
- [10] P. Antal, T. Péni, and R. Tóth, "Backflipping with miniature quadcopters by Gaussian-process-based control and planning," *IEEE Trans. Control Syst. Technol.*, vol. 32, no. 1, pp. 3–14, Jan. 2024.
- [11] G. Williams et al., "Information theoretic MPC for model-based reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Automat. (ICRA)*, Piscataway, NJ, USA: IEEE, 2017, pp. 1714–1721.
- [12] D. D. Fan, A.-a. Agha-mohammadi, and E. A. Theodorou, "Deep learning tubes for tube MPC," 2020, *arXiv:2002.01587*.
- [13] B. Amos, I. Jimenez, J. Sacks, B. Boots, and J. Z. Kolter, "Differentiable MPC for end-to-end planning and control," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 8299–8310.
- [14] R. Tao, S. Cheng, X. Wang, S. Wang, and N. Hovakimyan, "DiffTune-MPC: Closed-loop learning for model predictive control," *IEEE Robot. Automat. Lett.*, vol. 9, no. 8, pp. 7294–7301, Aug. 2024.
- [15] S. Meng, F. Meng, F. Zhang, Q. Li, Y. Zhang, and A. Zemouche, "Observer design method for nonlinear generalized systems with nonlinear algebraic constraints with applications," *Automatica*, vol. 162, 2024.
- [16] Z. Qiu, J. Yang, and L. Guo, "Stochastic stable attitude estimation algorithm using UKF with measurement loss," *IEEE/ASME Trans. Mechatron.*, vol. 27, no. 2, pp. 1059–1069, Apr. 2022.
- [17] M. Osman, M. W. Mehrez, M. A. Daoud, A. Hussein, S. Jeon, and W. Melek, "A generic multi-sensor fusion scheme for localization of autonomous platforms using moving horizon estimation," *Trans. Inst. Meas. Control*, vol. 43, no. 15, pp. 3413–3427, 2021.
- [18] A. Wenz and T. A. Johansen, "Moving horizon estimation of air data parameters for UAVs," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 56, no. 3, pp. 2101–2121, Jun. 2020.
- [19] S. Knüfer and M. A. Müller, "Nonlinear full information and moving horizon estimation: Robust global asymptotic stability," *Automatica*, vol. 150, 2023, Art. no. 110603.
- [20] J. B. Rawlings and B. R. Bakshi, "Particle filtering and moving horizon estimation," *Comput. Chem. Eng.*, vol. 30, no. 10–12, pp. 1529–1541, 2006.
- [21] W. Zhang, Z. Wang, C. Zou, L. Drugge, and M. Nybacka, "Advanced vehicle state monitoring: Evaluating moving horizon estimators and unscented Kalman filter," *IEEE Trans. Veh. Technol.*, vol. 68, no. 6, pp. 5430–5442, Jun. 2019.
- [22] S. Cheng et al., "DiffTune: Auto-tuning through auto-differentiation," *IEEE Trans. Robot.*, vol. 40, pp. 4085–4101, 2024.
- [23] C. D. McKinnon and A. P. Schoellig, "Estimating and reacting to forces and torques resulting from common aerodynamic disturbances acting on quadrotors," *Robot. Auton. Syst.*, vol. 123, 2020, Art. no. 103314.

- [24] C. V. Rao, J. B. Rawlings, and D. Q. Mayne, "Constrained state estimation for nonlinear discrete-time systems: Stability and moving horizon approximations," *IEEE Trans. Autom. Control*, vol. 48, no. 2, pp. 246–258, Feb. 2003.
- [25] J. Liu et al., "Simultaneous state and parameter estimation: the role of sensitivity analysis," *Ind. Eng. Chem. Res.*, vol. 60, no. 7, pp. 2971–2982, 2021.
- [26] U. Eren, A. Prach, B. B. Koçer, S. V. Raković, E. Kayacan, and B. Açıkmeşe, "Model predictive control in aerospace systems: Current state and opportunities," *J. Guid., Control, Dyn.*, vol. 40, no. 7, pp. 1541–1566, 2017.
- [27] W. Jin, Z. Wang, Z. Yang, and S. Mou, "Pontryagin differentiable programming: An end-to-end learning and control framework," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 7979–7992.
- [28] B. Amos and J. Z. Kolter, "OptNet: Differentiable optimization as a layer in neural networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2017, pp. 136–145.
- [29] Y. Wang and S. Boyd, "Fast model predictive control using online optimization," *IEEE Trans. Control Syst. Technol.*, vol. 18, no. 2, pp. 267–278, Mar. 2010.
- [30] B. Amos, L. Xu, and J. Z. Kolter, "Input convex neural networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2017, pp. 146–155.
- [31] A. Shapiro, D. Dentcheva, and A. Ruszczyński, *Lectures on Stochastic Programming: Modeling and Theory*, vol. 28. Philadelphia, PA, USA: SIAM, 2021.
- [32] L. Goldstein, "On the choice of step size in the Robbins-Monro procedure," *Statist. Probability Lett.*, vol. 6, no. 5, pp. 299–303, 1988.
- [33] J. Förster, "System identification of the Crazyflie 2.0 nano quadcopter," Bachelor's thesis, ETH Zurich, 2015.
- [34] R. Verschuere et al., "Acados—A modular open-source framework for fast embedded optimal control," *Math. Program. Comput.*, vol. 14, no. 1, pp. 147–183, 2022.



Her research interests include model predictive control, learning control, and its applications.



His research interests include model predictive control, learning control, cooperative control, and its applications in industrial mechatronics.

Dr. Li was a Guest Technical Editor for a Special Section in IEEE/ASME TRANSACTIONS ON MECHATRONICS. He serves as an Associate Editor of IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, and *ASME Journal of Dynamic Systems, Measurement and Control*.

Chenjing Meng (Graduate Student Member, IEEE) received the Bachelor of Engineering degree in automation from Chang'an University, Xi'an, China, in 2018, and the master's degree in naval architecture and ocean engineering, in 2021 from the School of Marine Science and Technology, Northwestern Polytechnical University, Xi'an, where she is currently working toward the Ph.D. degree in armament science and technology with the School of Marine Science and Technology.

Huiping Li (Senior Member, IEEE) received the B.Sc. and M.Sc. degrees in mechanical engineering and automatic control from the Northwestern Polytechnical University, Xi'an, China, in 2006 and 2009, respectively, and the Ph.D. degree in mechanical engineering from the University of Victoria, Victoria, BC, Canada, in 2013.

Since 2013, he has been with the School of Marine Science and Technology, Northwestern Polytechnical University, Xi'an, where he is currently a Full Professor. In 2013, he was a Postdoctoral Research Fellow with the Department of Mechanical Engineering, University of Victoria.